

Spark face-flux solver

A layout-independent mathematical algorithm

Derived from the active `hy_rk_getFaceFlux` call path

Purpose. Compute each directional face flux from a supplied stage state. The steps below specify the mathematics and its dependencies, so the computation can use any data layout, traversal order, or intermediate storage. All formulas needed for reconstruction, physical-state repair, the Riemann solve, artificial viscosity, and advected scalars are included.

Scope. The reference is the first, host `hy_rk_getFaceFlux` routine in `hy_rk_getFaceFlux.F90`, with the directly called `reconstruct`, `riemann`, `prim2con`, and `prim2flx` routines. All these files are in `source/physics/Hydro/HydroMain/Spark/`. This is a face-flux computation, not the subsequent flux-divergence update or Runge–Kutta stage combination. Boundary values and the stage state are supplied by the caller. Grid metadata access and index permutations only select cells and faces; they introduce no additional solver formula. Inactive hydrostatic adjustments are documented separately in Appendix A.

Inputs, notation, and outputs

Let $d \in \{1, \dots, D\}$ be an active coordinate direction with unit vector $\mathbf{e}_d$. A cell C has stage data

$$(\rho_C, \mathbf{u}_C, p_C, \gamma_C, e_C, \mathbf{B}_C, \psi_C, X_{1,C}, \dots, X_{N,C}, h_C). \quad (1)$$

e_C is specific internal energy; γ_C is the supplied sound-speed coefficient. X includes species fractions and other mass scalars. h_C is a supplied shock marker, or zero when absent. Magnetic fields and ψ are included only for GLM-MHD. Velocity and magnetic vectors always have three components, even when $D < 3$.

The reconstructed quantities are the individual components of

$$Q_C = (\rho_C, u_{x,C}, u_{y,C}, u_{z,C}, p_C, \gamma_C, \varepsilon_C, \mathbf{B}_C, \psi_C, X_{1,C}, \dots, X_{N,C}), \quad \varepsilon_C = \rho_C e_C. \quad (2)$$

Omit $\mathbf{B}, \psi$ for hydrodynamics. Reconstruct $\varepsilon = \rho e$ directly, rather than reconstructing e and then multiplying by the reconstructed density. No equation of state is evaluated in this path.

For each requested face between adjacent cells A and $B = A + \mathbf{e}_d$, return fluxes of mass, three momenta, total energy, optional magnetic fields and ψ , and each conserved mass scalar ρX_a . The positive orientation is from A to B . Fluxes are per unit area; this routine multiplies by neither face area nor time step.

Step 1. Compute the cell flattening coefficient

If flattening is disabled, set $f_C = 1$ for every required cell. Otherwise use the original stage pressure, velocity, and γ . For each active direction define

$$\Delta p_{C,d} = |p_{C+e_d} - p_{C-e_d}|, \quad (3)$$

$$b_{C,d} = \frac{\Delta p_{C,d}}{\max(\eta, |p_{C+2e_d} - p_{C-2e_d}|)}, \quad \eta = 10^{-99}, \quad (4)$$

$$Z_{C,d} = \frac{\Delta p_{C,d}}{\gamma_C p_C}, \quad (5)$$

$$a_{C,d} = \max \left(0, \min \left(1, \frac{0.85 - b_{C,d}}{0.85 - 0.75} \right) \right). \quad (6)$$

Then apply the compression test and combine directions:

$$\tilde{f}_{C,d} = \begin{cases} \max \left(a_{C,d}, \min \left(1, \frac{0.75 - Z_{C,d}}{0.75 - 0.25} \right) \right), & u_{d,C+e_d} < u_{d,C}, \\ 1, & \text{otherwise,} \end{cases} \quad (7)$$

$$f_C = \min_{d=1,\dots,D} \tilde{f}_{C,d}. \quad (8)$$

The compression predicate compares the forward neighbor with the current cell, exactly as in the reference. The maximum with $a_{C,d}$ keeps $\tilde{f}$ nonnegative even if the second term is negative. $f = 0$ produces constant reconstruction and $f = 1$ retains WENO before the monotonicity test. The same f_C is used for all variables and all reconstruction directions at that cell. If a flattening diagnostic field exists, it receives f_C .

Step 2. Select the geometric reconstruction stencil

For a fixed cell C , direction d , and one component q of Q , write

$$q_k = q(C + k e_d), \quad k = -2, -1, 0, 1, 2. \quad (9)$$

Compute both $q_C^{+,d}$, the state approaching the cell's positive face, and $q_C^{-,d}$, the state approaching its negative face. Apply Steps 3–4 independently to every component, including γ , ε , and all species and mass scalars. The reconstruction is componentwise in these primitive variables; there is no characteristic projection or normal/tangential rotation of stored vectors.

The formulas assume the uniform directional stencil used by the code. Although a cell spacing is passed to reconstruction, it does not appear in its executable arithmetic.

Step 3. Form WENO-Z candidates and weights

For the positive face form

$$P_1 = \frac{1}{3}q_{-2} - \frac{7}{6}q_{-1} + \frac{11}{6}q_0, \quad (10)$$

$$P_2 = -\frac{1}{6}q_{-1} + \frac{5}{6}q_0 + \frac{1}{3}q_1, \quad (11)$$

$$P_3 = \frac{1}{3}q_0 + \frac{5}{6}q_1 - \frac{1}{6}q_2. \quad (12)$$

For the negative face form

$$M_1 = -\frac{1}{6}q_{-2} + \frac{5}{6}q_{-1} + \frac{1}{3}q_0, \quad (13)$$

$$M_2 = \frac{1}{3}q_{-1} + \frac{5}{6}q_0 - \frac{1}{6}q_1, \quad (14)$$

$$M_3 = \frac{11}{6}q_0 - \frac{7}{6}q_1 + \frac{1}{3}q_2. \quad (15)$$

Use the same three smoothness indicators for both faces:

$$\beta_1 = \frac{13}{12}(q_{-2} - 2q_{-1} + q_0)^2 + \frac{1}{4}(q_{-2} - 4q_{-1} + 3q_0)^2, \quad (16)$$

$$\beta_2 = \frac{13}{12}(q_{-1} - 2q_0 + q_1)^2 + \frac{1}{4}(q_{-1} - q_1)^2, \quad (17)$$

$$\beta_3 = \frac{13}{12}(q_0 - 2q_1 + q_2)^2 + \frac{1}{4}(3q_0 - 4q_1 + q_2)^2, \quad (18)$$

$$\tau_5 = |\beta_1 - \beta_3|, \quad \epsilon_W = 10^{-36}. \quad (19)$$

For $\sigma \in \{+, -\}$, define

$$(g_1^+, g_2^+, g_3^+) = (0.1, 0.6, 0.3), \quad (g_1^-, g_2^-, g_3^-) = (0.3, 0.6, 0.1), \quad (20)$$

$$\alpha_k^\sigma = g_k^\sigma \left(1 + \frac{\tau_5}{\beta_k + \epsilon_W} \right), \quad \omega_k^\sigma = \frac{\alpha_k^\sigma}{\sum_{\ell=1}^3 \alpha_\ell^\sigma}. \quad (21)$$

The WENO-Z correction uses exponent one. The weights depend on the particular reconstructed variable; weights for pressure, density, velocity, and species are generally different.

Step 4. Combine, flatten, and enforce monotonicity

$$\widehat{q}_C^+ = \sum_{k=1}^3 \omega_k^+ P_k, \quad \widehat{q}_C^- = \sum_{k=1}^3 \omega_k^- M_k, \quad (22)$$

$$q_C^+ = f_C \widehat{q}_C^+ + (1 - f_C) q_0, \quad q_C^- = f_C \widehat{q}_C^- + (1 - f_C) q_0. \quad (23)$$

If $(q_C^+ - q_0)(q_0 - q_C^-) \leq 0$, replace *both* values by q_0 . This test occurs after flattening and independently for each component. It requires both face values from the same cell even when only one is needed at the face currently being evaluated.

Step 5. Assemble and repair the two face states

At the face between A and $B = A + \mathbf{e}_d$,

$$V_L = Q_A^{+,d}, \quad V_R = Q_B^{-,d}. \quad (24)$$

For each side $s \in \{L, R\}$ apply

$$\rho_s \leftarrow \max(\rho_{\min}, \rho_s), \quad p_s \leftarrow \max(p_{\min}, p_s). \quad (25)$$

Here $\rho_{\min}$ and $p_{\min}$ are configured floors. For each *species* $a = 1, \dots, N_{\text{sp}}$ compute

$$Y_{a,s} = \max(X_{\min}, \min(1, X_{a,s})), \quad X_{a,s} \leftarrow \frac{Y_{a,s}}{\sum_{b=1}^{N_{\text{sp}}} Y_{b,s}}. \quad (26)$$

This normalization is omitted if there are no species. Other mass scalars are not clipped or normalized. The repair does not adjust $\varepsilon_s, \gamma_s, \mathbf{u}_s, \mathbf{B}_s$, or ψ_s , and does not recompute energy after flooring density or pressure. A normalized species fraction can fall below $X_{\min}$; clipping precedes normalization.

Set the face shock predicate using the original two cell markers:

$$H_{AB} = (h_A \neq 0) \vee (h_B \neq 0). \quad (27)$$

This predicate is separate from the flattening sensor. The shock markers are supplied data, not computed by this routine.

Step 6. Begin the Riemann solve and determine wave speeds

All quantities in Steps 6–10 come from the repaired face states. Let $u_{n,s} = \mathbf{u}_s \cdot \mathbf{e}_d$; in MHD let $B_{n,s} = \mathbf{B}_s \cdot \mathbf{e}_d$. In hydrodynamics magnetic terms are zero. Compute

$$a_s^2 = \frac{\gamma_s p_s}{\rho_s}. \quad (28)$$

The Riemann routine first sets its error flag to zero. If any side has $0 < \rho_s < \tau$ or $0 < p_s < \tau$, where τ is its tiny threshold, return zero Riemann flux. Otherwise, if either $a_s^2 < 0$, return with error flag one. The latter path leaves the flux undefined; the face-flux caller does not check the flag. Thus the following normal path presumes admissible inputs and nonzero denominators. No extra numerical guards are part of the reference arithmetic.

For hydrodynamics $c_{f,s} = \sqrt{a_s^2}$. For MHD,

$$b_s^2 = \frac{|\mathbf{B}_s|^2}{\rho_s}, \quad c_{f,s} = \sqrt{\frac{1}{2} \left(a_s^2 + b_s^2 + \sqrt{(a_s^2 + b_s^2)^2 - 4a_s^2 B_{n,s}^2 / \rho_s} \right)}. \quad (29)$$

$$S_L = \min(u_{n,L} - c_{f,L}, u_{n,R} - c_{f,R}), \quad S_R = \max(u_{n,L} + c_{f,L}, u_{n,R} + c_{f,R}). \quad (30)$$

The local signal-speed diagnostic is $\max(|S_L|, |S_R|)$. It is not used further by this face-flux caller.

Step 7. Compute conservative states and physical fluxes

Use $\mathbf{m}_s = \rho_s \mathbf{u}_s$ and

$$E_s = \frac{1}{2} \rho_s |\mathbf{u}_s|^2 + \varepsilon_s + \frac{1}{2} |\mathbf{B}_s|^2, \quad P_s = p_s + \frac{1}{2} |\mathbf{B}_s|^2. \quad (31)$$

For hydrodynamics the magnetic terms vanish. The conservative states are $U_s = (\rho_s, \mathbf{m}_s, E_s)$, extended by $(\mathbf{B}_s, \psi_s)$ for MHD. No ψ energy term is included.

For $j \in \{x, y, z\}$ and δ_{jd} the Kronecker delta, the assigned physical flux components are

$$(F_s)_\rho = \rho_s u_{n,s}, \quad (32)$$

$$(F_s)_{m_j} = \rho_s u_{n,s} u_{j,s} - B_{n,s} B_{j,s} + P_s \delta_{jd}, \quad (33)$$

$$(F_s)_E = (E_s + P_s) u_{n,s} - B_{n,s} (\mathbf{u}_s \cdot \mathbf{B}_s). \quad (34)$$

For MHD also set

$$(F_s)_{B_j} = u_{n,s} B_{j,s} - u_{j,s} B_{n,s}, \quad (F_s)_{B_n} = 0, \quad (F_s)_\psi = 0. \quad (35)$$

These are intermediate physical fluxes. The normal magnetic and ψ numerical fluxes receive separate GLM values in Step 10.

Step 8. Compute the HLL state and contact quantities

$$U^H = \begin{cases} U_L, & S_L > 0, \\ \frac{S_R U_R - S_L U_L - F_R + F_L}{S_R - S_L}, & S_L \leq 0 \text{ and } S_R \geq 0, \\ U_R, & \text{otherwise.} \end{cases} \quad (36)$$

For MHD extract $\mathbf{B}^* = \mathbf{B}^H$, $B_n^* = B_n^H$, and $\mathbf{u}^H = \mathbf{m}^H / \rho^H$.

$$q^* = \frac{\rho_R u_{n,R} (S_R - u_{n,R}) - \rho_L u_{n,L} (S_L - u_{n,L}) + P_L - P_R - B_{n,L}^2 + B_{n,R}^2}{\rho_R (S_R - u_{n,R}) - \rho_L (S_L - u_{n,L})}. \quad (37)$$

For $s = L, R$, define $N_s = S_s - u_{n,s}$ and $D_s = S_s - q^*$. The common star pressure, evaluated from the left side, is

$$p^* = \rho_L N_L (q^* - u_{n,L}) + P_L - B_{n,L}^2 + (B_n^*)^2. \quad (38)$$

For hydrodynamics omit both magnetic contributions and the HLL magnetic extraction. The HLL conservative state itself is not otherwise needed for the hydrodynamic star states.

Step 9. Compute the two star states

For each side $s = L, R$, assign

$$\rho_s^* = \frac{\rho_s N_s}{D_s}, \quad (39)$$

$$(m_n)_s^* = \rho_s^* q^*, \quad (40)$$

$$(m_t)_s^* = \frac{\rho_s u_{t,s} N_s - B_n^* B_t^* + B_{n,s} B_{t,s}}{D_s}, \quad t \neq d, \quad (41)$$

$$E_s^* = \frac{E_s N_s + p^* q^* - P_s u_{n,s}}{D_s} - \frac{B_n^* (\mathbf{B}^* \cdot \mathbf{u}^H) - B_{n,s} (\mathbf{B}_s \cdot \mathbf{u}_s)}{D_s}. \quad (42)$$

In hydrodynamics omit all magnetic terms. For MHD the star magnetic field on *both* sides is $\mathbf{B}^*$.

These formulas specify all mass, momentum, energy, and magnetic components used below. No star ψ value is required by the layout-independent algorithm: compute the final ψ flux directly in Step 10. The reference does not assign the star ψ components before its vector arithmetic, although it overwrites their resulting flux afterward. An implementation can avoid that unassigned intermediate by applying the following HLLC formulas only to the needed components.

Step 10. Select the numerical Riemann flux

Apply the HLLC branches in the indicated order:

$$F^R = \begin{cases} F_L, & S_L \geq 0, \\ F_L + S_L(U_L^* - U_L), & S_L < 0 \text{ and } q^* \geq 0, \\ F_R + S_R(U_R^* - U_R), & S_L < 0, q^* < 0, S_R \geq 0, \\ F_R, & \text{otherwise.} \end{cases} \quad (43)$$

If hybrid solving is enabled and H_{AB} is true, replace this by

$$F^R \leftarrow \begin{cases} F_L, & S_L > 0, \\ F_R, & S_L \leq 0 \text{ and } S_R < 0, \\ \frac{S_R F_L - S_L F_R + S_R S_L (U_R - U_L)}{S_R - S_L}, & \text{otherwise.} \end{cases} \quad (44)$$

For MHD with configured cleaning speed c_h , compute

$$B_n^G = \frac{1}{2}(B_{n,L} + B_{n,R}) - \frac{\psi_R - \psi_L}{2c_h}, \quad (45)$$

$$\psi^G = \frac{1}{2}(\psi_L + \psi_R) - \frac{1}{2}c_h(B_{n,R} - B_{n,L}), \quad (46)$$

$$(F^R)_{B_n} \leftarrow \psi^G, \quad (F^R)_\psi \leftarrow c_h^2 B_n^G. \quad (47)$$

These GLM states do not replace the input fields used in Steps 6–9. The reference computes them earlier; placing them here preserves their dependencies and the final result without prescribing storage.

Step 11. Add artificial viscosity from original cell states

Return to the *unreconstructed* stage states in cells A, B . With configured coefficient C_v , compute

$$\nu_{AB} = C_v \max(u_{d,A} - u_{d,B}, 0). \quad (48)$$

There are no transverse velocity differences, cell-spacing factors, or time-step factors in this coefficient. Form the original-cell energy used specifically by viscosity:

$$E_C^v = \frac{1}{2} \rho_C |\mathbf{u}_C|^2 + \varepsilon_C, \quad C \in \{A, B\}. \quad (49)$$

In MHD E_C^v still excludes magnetic energy, unlike E_s in the Riemann solve. Define the viscosity state

$$W_C^v = (\rho_C, \rho_C \mathbf{u}_C, E_C^v) \quad \text{or} \quad W_C^v = (\rho_C, \rho_C \mathbf{u}_C, E_C^v, \mathbf{B}_C, \psi_C) \text{ for MHD.} \quad (50)$$

For those components set

$$\boxed{F^v = F^R + \nu_{AB}(W_A^v - W_B^v)}. \quad (51)$$

In particular the final magnetic and cleaning fluxes now include

$$(F^v)_{B_n} = \psi^G + \nu_{AB}(B_{d,A} - B_{d,B}), \quad (52)$$

$$(F^v)_\psi = c_h^2 B_n^G + \nu_{AB}(\psi_A - \psi_B). \quad (53)$$

Thus the GLM overwrite is followed by viscosity, rather than being the last operation of the complete face-flux procedure. A zero-flux early return from the Riemann solve is also followed by this viscosity step.

Step 12. Compute mass-scalar fluxes and emit the face result

Let $M = (F^v)_\rho$, the mass flux *after* viscosity. For every species and every other mass scalar,

$$X_a^{\text{up}} = \begin{cases} X_{a,L}, & M > 0, \\ X_{a,R}, & M \leq 0, \end{cases} \quad \boxed{F_{\rho X_a} = M X_a^{\text{up}}}. \quad (54)$$

The fractions are the repaired, reconstructed face fractions from Step 5, not the original cell fractions. There is no separate scalar viscosity term. For zero mass flux the right state is selected and the scalar flux is zero. Normalized species give $\sum_a F_{\rho X_a} = M$ in exact arithmetic, with the sum restricted to species.

Emit F^v and these scalar fluxes for the geometrically identified face, using the same component orientation in every direction. Saving the face result introduces no further arithmetic or rescaling.

Dependencies for an implementation with different storage

The stage state must remain available unchanged through the viscosity calculation. Repaired face states are separate mathematical values even if their actual storage is temporary or their expressions are fused.

1. Form $\varepsilon_C = \rho_C e_C$ and optional flattening sensors from original stage data.
2. For each requested face direction, evaluate both reconstructions in its two adjacent cells, including each cell's monotonicity test.
3. Assemble the approaching states and apply floors and species normalization.
4. Evaluate the Riemann flux, hybrid selection, and GLM overwrite.
5. Add viscosity using original adjacent-cell data.
6. Use the corrected mass flux to choose advected fractions, then emit all flux components for that face.

Reconstruction may be evaluated once per cell and reused on its two faces, or recomputed per face. Candidate polynomials and nonlinear weights need not be stored. The HLL state need only retain the components required by subsequent magnetic formulas. Intermediate vector operations may instead be evaluated component by component. These choices do not change the mathematical dependency graph.

Required neighborhood. At the face between A and $B = A + \mathbf{e}_d$, WENO needs the six-cell union $A - 2\mathbf{e}_d, \dots, A + 3\mathbf{e}_d$. If flattening is enabled, it also needs pressure out to two cells in every active direction around each of A, B , the forward velocity in every direction, and each cell's pressure and γ . Viscosity needs only A, B . Ghost or boundary data must supply these neighborhoods. For requested cell limits $\ell_d, \dots, r_d$, the reference emits directional faces between cells $i - 1$ and i for $i = \ell_d, \dots, r_d + 1$, at all transverse cells in the requested limits. Hence the outer face reconstruction requires data through $\ell_d - 3$ and $r_d + 3$ in that direction.

Parameter correspondence.

Mathematical input	Reference configuration
Flattening enabled	hy_flattening
Hybrid HLLC enabled	hy_hybridRiemann
$\rho_{\min}, p_{\min}, X_{\min}$	hy_smallldens, hy_smallpres, hy_smallX
τ, c_h, C_v	hy_tiny, hy_C_hyp, hy_cvisc
MHD components enabled	SPARK_GLM
Number of species / other scalars	NSPECIES / NMASS_SCALARS

The literal 10^{-99} in the flattening denominator may underflow in some real precisions. The source also contains unguarded divisions and an unchecked Riemann error return. This document supplies the normal algorithm and identifies those existing limits; it does not introduce a new fallback policy. Changes in evaluation order can change floating point rounding even when the mathematical algorithm is identical.

Appendix A. Inactive hydrostatic pressure arithmetic

The host face-flux routine defines hydrostatic helpers, but their calls are commented out. They do *not* contribute to Steps 1–12. For completeness, their arithmetic about a reconstruction cell C is given here. Let $C_- = C - \mathbf{e}_d$ and $C_+ = C + \mathbf{e}_d$. If gravitational potential ϕ is available, define

$$A_- = \phi_C - \phi_{C_-}, \quad A_+ = \phi_{C_+} - \phi_C. \quad (55)$$

Otherwise, with directional acceleration g_d and spacing Δx_d ,

$$A_- = \frac{1}{2}(g_{d,C_-} + g_{d,C})\Delta x_d, \quad A_+ = \frac{1}{2}(g_{d,C} + g_{d,C_+})\Delta x_d. \quad (56)$$

The preparation helper modifies two neighboring pressures:

$$p_{C_-} \leftarrow p_{C_-} - \frac{1}{2}(\rho_{C_-} + \rho_C)A_-, \quad (57)$$

$$p_{C_+} \leftarrow p_{C_+} + \frac{1}{2}(\rho_C + \rho_{C_+})A_+. \quad (58)$$

The reset helper adds the subtracted term back to p_{C_-} and subtracts the added term from p_{C_+} . With gravity disabled both helpers return without modification. Gravity data copied by the host input preparation are otherwise unused by its active flux algorithm.

The same file also defines a distinct offloaded entry point. It aborts when flattening is enabled and includes active hydrostatic helper calls. It is therefore not assumed to be an interchangeable execution of the host path described here. Porting the host mathematics should follow Steps 1–12 rather than importing those additional calls implicitly.

Appendix B. Arithmetic coverage and source map

All files below are in `source/physics/Hydro/HydroMain/Spark/`.

Source	Covered mathematical operation
<code>hy_rk_getFaceFlux.F90</code>	setPencil : $\varepsilon = \rho e$ and geometric input selection; flattening : Step 1; ensurePhysicalState : Step 5; avisc : Step 11; mscalarFluxes : Step 12.
<code>reconstruct.F90</code>	Steps 2–4, the active reconstruct routine. The standalone minmod function is not called by it.
<code>riemann.F90</code>	Steps 6, 8–10, the active riemann routine.
<code>prim2con.F90</code> , <code>prim2flx.F90</code>	Step 7, called by riemann and included through riemann.F90 .
<code>setLoop</code> , <code>saveFluxes</code> , <code>grid metadata</code>	Geometric direction selection and face output placement; no flux arithmetic.
<code>setHSE</code> , <code>resetHSE</code>	Inactive host helpers, Appendix A.

The largest-direction allocation, pointer reindexing, pencil copies, OpenMP scheduling, and rolling reuse of the previous positive face state are storage and traversal choices. They are replaced here by explicit cell and face identities. No Eulerian time update, shock-marker generation, EOS operation, or boundary-condition construction is called by the active flux routine, so none is inferred as part of its arithmetic.